

SQL Injection Attack

Import

The SQL Injection attack is one of the most common attacks on web applications. It is a *code injection* technique that exploits vulnerabilities in the interaction between **web applications** and **database servers**. The vulnerability occurs when user-entered data **is not properly validated** within the applications before being sent to the backend database.

Many web applications take data from users and then use it directly to **construct SQL queries** to retrieve information from the database. They also use SQL queries to **store** information in the database. However, when SQL queries **are not carefully** constructed, vulnerabilities can arise that allow **SQL injection attacks**. In this lab, we will use a **web application** that includes common mistakes made by many developers. The goal of the exercise is to highlight ways to exploit these mistakes, demonstrate the damage they can cause, and finally examine techniques that can help prevent these attacks.

Laboratory environment

This exercise has been tested on the **SEEDUbuntu - Ubuntu 16.04 virtual machine**. For the needs of the exercise, a **web application is used**, which has been developed for this purpose.

Web Server

The application is hosted on the **Apache Web Server**, which is installed on the virtual machine and runs automatically at startup.

It uses *virtual hosting* to host multiple websites (or URLs) on the same machine. Configuration instructions are contained in the 000-default.conf file located in the "/etc/apache2/sites-available" directory.

Within the configuration file, each web application has a VirtualHost block, which specifies the URL for the website and the directory on the file system that contains the code for the website, according to the following format:

```
<VirtualHost *>
    ServerName http://www.example1.com
    DocumentRoot /var/www/Example_1/
</VirtualHost>

<VirtualHost *>
    ServerName http://www.example2.com
    DocumentRoot /var/www/Example_2/
</VirtualHost>
```

The directory in which the specific application is installed and the address
The URL to access it (via the virtual machine's browser) is as follows:

URL: http://www.SEEDLabSQLInjection.com
Directory: /var/www/SQLInjection

The above URL is **only available from within the virtual machine** because the **/etc/hosts** file has been modified to map the *domain name* of each URL to the virtual machine's local IP address (**127.0.0.1**).

Having access to the above directories we can make modifications to the application code. If any **change is made**, either to the **configuration** or to the **code** of an application, the *Apache Server* must be restarted for this change to take effect. The following commands are used to stop and start the Apache Server respectively:

```
$ sudo service apache2 stop
$ sudo service apache2 start
```

or

```
$ sudo service apache2 restart
```

Database Server

The application **database** is hosted on a **MySQL Server**, which is already installed and runs automatically when the virtual machine starts.

The database administrator username is **root** and the password is **seedubuntu**. The connection to the **MySQL Server** console can be done using the following command:

```
$ mysql -u root -pseedubuntu
```

Web Application

The web application we use to demonstrate the activities of the exercise is a simple **employee management** application hosted at **www.SEEDLabSQLInjection.com**.

Employees can use it to **view** and **update** their personal information in a **database**. The application supports two main categories of users:

- The **administrator (Admin)** is a user with administrative privileges, who can manage the profile information of all employees.
- **Employee** is a regular user, who can view or update just some of his own profile information.

All the initial information already entered into the database is described in the table below:

Name	Employee ID	Password	Salary	Birthday	SSN	Nickname	Email	Address	Phone#
Admin	99999	seedadmin	400000	3/5	43254314				
Alice	10000	seedalice	20000	9/20	10211002				
Boby	20000	seedboby	50000	4/20	10213352				
Ryan	30000	seedryan	90000	4/10	32193525				
Samy	40000	seedsamy	40000	1/11	32111111				
Ted	50000	seedted	110000	11/3	24343244				

Activity 1: Familiarity with SQL commands

The goal of this activity is to familiarize yourself with SQL commands using the MySQL Server console. The database we are using is called **Users** and contains a table called **credential**. The table stores the personal information (e.g. *eid*, *password*, *salary*, *ssn*, etc.) of each employee. Loading the **Users** database into the MySQL Server console is done using the following command:

```
mysql> use Users;
```

To see in the console which tables exist in the **Users** database, you can use the following command, which will print all the tables of the selected database in the output.

```
mysql> show tables;
```

The purpose of **Activity 1** is, after connecting to the **Users database**, to execute the appropriate **SQL queries** to perform the following actions:

- **Activity 1.1: Printing table information**

Print (to the terminal) information about the structure of the **credential** table (i.e. its fields and corresponding types).

- **Activity 1.2: Printing user information**

Print (in the terminal) all the contents of the **credential table**. Then print all the information only for the employee named **Samy**.

- **Activity 1.3: Introducing new users**

Enter two new users (with all fields filled in **except for the password**), which will be used for the attacks. One of the new users should have **name**, your *first name*.

- **Activity 1.4: Password update for new users**

In this activity, you are asked to choose **passwords** for the two new users and update their records in the database. For security reasons, the **password** field is not stored in a readable (plain) format. What is stored is the **digest**, which is generated by a **hash algorithm**. This means that, before updating the records, you will need to find the **hash** values of the selected **passwords**. The algorithm used in this application is

SHA1.

*In your report, in addition to explaining your actions, you should include screenshots of the commands you used and their results. After each insert or update action, you should execute the appropriate **SELECT query** that will show the success of the action.*

Activity 2: SQL Injection attack on SELECT statements

SQL **Injection** is basically a technique through which attackers can execute their own (malicious) **SQL commands**. Through malicious **SQL commands**, attackers can **steal** information about the victims from the database or even worse, they can **make changes** to the database. This particular employee management application contains common mistakes that developers make.

In Activity 2, we will use the login page of the **www.SEEDLabSQLInjection.com application**. The login page (*Figure 1*) asks users to enter **their username** and **password**. The application authenticates the user based on these two pieces of information, so only employees who know their passwords are allowed to log in. Your mission, as an attacker, is **to log in** to the web application **WITHOUT** knowing any employee's credentials.

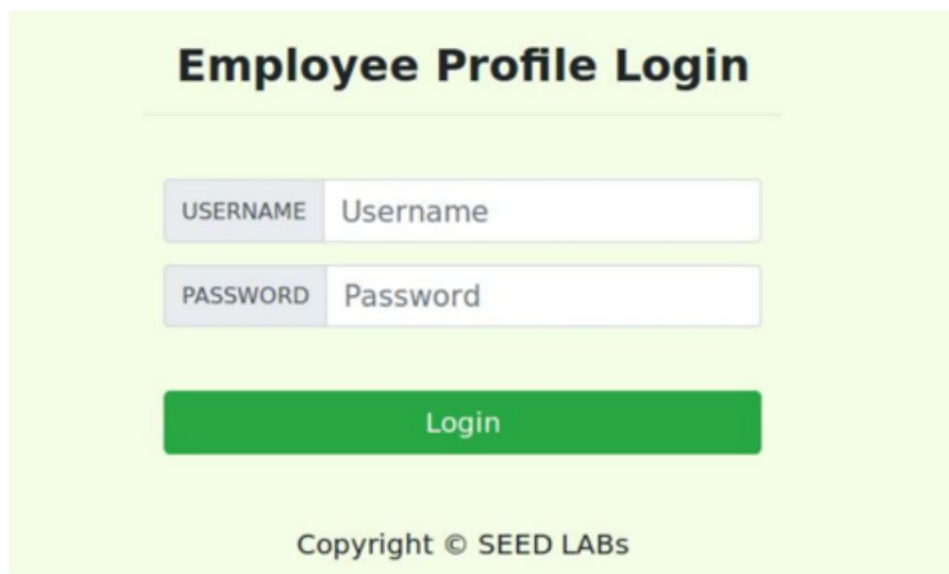


Figure 1. Login page

To help you with this activity, here is a detailed explanation of how authentication is implemented in the application (at the code level):

The **PHP** code used to perform user verification is located in the **unsafe_home.php file**, which is stored in the **/var/www/SQLInjection directory**. The following code snippet shows how to verify users.

```

$input_uname = $_GET['username'];
$input_pwd = $_GET['Password'];
$hashed_pwd = sha1($input_pwd);
...
$sql = "SELECT id, name, eid, salary, birth, ssn, address, email,
        nickname, Password
        FROM credential
        WHERE name= '$input_uname' and Password='$hashed_pwd'";
$result = $conn->query($sql);

// The following is Pseudo Code
if(id != NULL) {
    if(name=='admin') {
        return All employees information;
    } else if (name !=NULL){
        return employee information;
    }
} else {
    Authentication Fails;
}

```

The above SQL statement selects the personal information of the employees, such as *id*, *name*, *salary*, *ssn*, etc., from the **credential** table. The SQL statement uses two variables **input_uname** and **hashed_pwd**, where **input_uname** contains the string that the users type in the **username** field of the login page, while **hashed_pwd** contains the hashed value (using the sha1 algorithm) **of the password** that the user types. The program checks whether any record matches the provided *username* and *password*. If there is a match, the user is successfully authenticated and gains access to the corresponding information. If there is no match, the authentication fails.

With the above in mind, you should complete the following activities:

• Activity 2.1: SQL Injection attack on the login page.

The purpose of **ACTIVITY 2.1** is to **log in** to the web application **as an administrator** from the **login page**, so that you can view all employee information. We assume that you only know the administrator account name (**admin**), but **do NOT** have the password. You need to decide what to type in the **Username** and **Password** fields for the attack to succeed.

In your report you should include screenshots of the input you typed into the fields and the result of the attack. Also, explain why the attack was successful, based on the above php code snippet describing the login process.

• Activity 2.2: SQL Injection attack from the command line.

The purpose of **Activity 2.2** is to repeat **activity 2.1**, but **WITHOUT** use the **login page**. Instead, you can use **command line** tools that can send **HTTP requests**, such as curl program . You should note that if you want to include **multiple parameters** in **HTTP requests**, you must enclose the URL **and** the parameters in **single quotes** ('). Otherwise, special characters used to separate parameters (such as &) will be interpreted by the shell program, changing the meaning of the command. The following **example** shows how to send a **GET HTTP** request

request to our application, with two parameters (username and password) attached:

```
$ curl 'www.SEEDLabSQLInjection.com/unsafe_home.php?username=alice&password=111'
```

Also, if you need to include **special characters** in the **username** fields (**username**) or **password (password)**, you must encode them correctly (**hex ASCII**) or the meaning of your requests will change. For example, if you want to include a single quote (') in these fields, you must instead of This means you can use the **%27** notation . If you want to include whitespace, you should use the **%20** notation.

In this activity, you will need to handle **HTTP encoding when sending requests using curl**. In your report, you will need to include screenshots of the command you typed and the result of the attack.

To better handle the HTML content you receive in response to the terminal, you can use a **text-web browser such as Lynx1. This tool can be installed on the virtual machine using the command:*

```
$ sudo apt install lynx
```

Once installed, you can use it by appropriately modifying the command in the above example as follows:

```
$ curl 'www.SEEDLabSQLInjection.com/unsafe_home.php?....' | lynx --stdin
```

¹ <https://lynx.invisible-island.net>

- **Activity 2.3: Executing Multiple SQL Statements.** In the two attacks

above, we can only **bypass** control mechanisms or **steal** information from the database. It would be even more dangerous if we could **modify the database** using the same vulnerability on the login page. One way to do this is to use the **SQL Injection** attack to turn **one SQL** statement into **two**, with the second being the **update** or **delete** statement.

(delete statement). In SQL, the **semicolon (;)** is used to separate two **SQL**

statements. The purpose of **Activity 2.3** is to use the **username** and/or **password** fields of the login page to **delete** a record or **update** any field in the database. Also, attempt – again by executing multiple SQL statements from the login page – to retrieve a user's password (essentially, its hash value).

*In your report, describe how you can use the login page to execute multiple **SQL commands**. Test the attack and record the results. Explain the success or failure of the attack, considering the php code snippet that describes the **login process**.*

- **Activity 2.4: Password recovery (rainbow attack)**

From Activity 2.3, you will either have managed to recover a user's password (if the attack is successful) or not (if the attack cannot be carried out). In any case, regardless of the outcome of Activity 2.3, you assume that the attack is successful and that you can recover the **fingerprints** (hash values) of all users' passwords (which you can have anyway, for the purposes of this Activity, from Activity 1.2). Theoretically, it is not possible to recover the passwords themselves from the fingerprints since hash functions are irreversible. But is this always the case? An attacker can pre-compute the fingerprints of words/strings that are likely to be chosen by a user, thus constructing a super-file consisting of possible passwords and, for each of them, its fingerprint (for various hash functions). In this way, he can check if a password fingerprint exists in this super-file: if so, this means that he directly recognizes the password. This is a **“rainbow” attack**.

Using the hash values of the passwords you have recovered (including passwords you have created), check whether the **rainbow** attack can fully reveal them. For this purpose

For this you can utilize appropriate online tools, such as the websites <https://hashes.com>, <https://crackstation.net/>.

In your report, describe (using appropriate screenshots) how you performed the Rainbow attack on the (hashed) passwords you have recovered. Comment on the results you obtained.

Activity 3: SQL Injection attack on UPDATE statements

If an **SQL Injection** vulnerability occurs in an **UPDATE statement**, the potential damage will be more severe because attackers can use the vulnerability to **modify** a database. In the **Employee Management** application, there is a page for **editing** user profiles (Figure 2), which allows employees to update their profile information, including *nickname*, *email*, *address*, *phone number*, and *password*. To access this page, employees must first log in and then select **Edit Profile**.

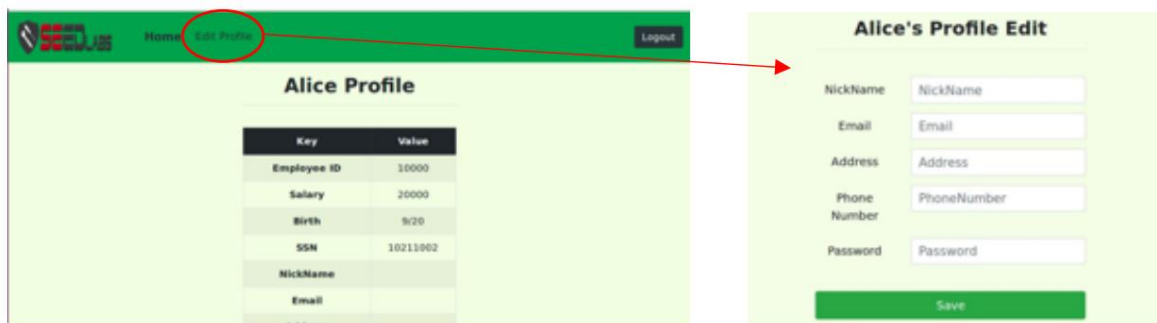


Figure 2. Profile edit page

When a user saves **their** information via the **Edit** page **Profile**, the following SQL UPDATE query will be executed :

```
$hashed_pwd = sha1($input_pwd);
$sql = "UPDATE credential SET
    nickname=' $input_nickname',
    email=' $input_email',
    address=' $input_address',
    Password=' $hashed_pwd',
    PhoneNumber=' $input_phonenumber'
    WHERE ID=$id;";
$conn->query($sql);
```

The **PHP** code used to update employee profile information is located in the **unsafe_edit_backend.php** file. The **PHP** file it is also located in the **/var/www/SQLInjection** directory.

With the above in mind, you should complete the following activities:

- **Activity 3.1: Modifying your own details.**

As you can see on the **"Edit Profile"** page, employees can only update their *nickname*, *email*, *address*, *phone number*, and *password*. They are not allowed to change other details, such as their **salary**. Let's say **you**

You are a disgruntled employee because your boss (**Boby**) did not increase your salary this year.

The purpose of **Activity 3.1** is, after logging in with **your account** (which you created in activity **1.3-1.4**), to increase **ONLY** your own **salary** to whatever amount you wish, by exploiting the SQL Injection vulnerability on the *Edit* page.

Profile. You should be careful that the query that will be executed **does not affect them**. salaries of other employees.

*In your report you should explain how you will carry out the attack and include screenshots of the **input** you typed into the corresponding fields. You should also include screenshots of the **profile** your **page** before and after the attack, to prove its success.*

- **Activity 3.2: Modifying other users' data.**

After increasing your own salary, you decide to help another **colleague**. (the **second user** you created in activity **1.3-1.4**) and then punish your boss **Boby**.

The purpose of **Activity 3.2** is, after logging in **to your account**, on the *Edit Profile* page , to perform the following actions: (1) **increase** your colleague's salary to any amount you wish, (2) **decrease** the salary of user **Boby** at **\$1**, (3) change user Bobby's **phone** . Attacks are not necessary be done in one action, but in as many as needed. As before, you should be careful that the queries that will be executed **do not affect** the data of other employees.

*In your report you should explain how you will carry out the attack and include screenshots of the **input** you typed into the corresponding fields. You should also include screenshots of the **profiles** **pages** of your **colleague** and **Boby** before and after the attack, to prove its success.*

- **Activity 3.3: Modifying other users' passwords.**

After changing Bobby's salary , you are still unhappy, so you want to change his password and then be able to log in **to his account** and cause further damage. The purpose of **Activity 3.3** is , after logging in **to your account**, on the *Edit Profile* page , to **change ONLY** Bobby's **password** to something you know. As before, you must be careful that the query you run **does not affect** the passwords of other employees.

*In your report, describe how you can achieve this and include screenshots of the input you typed into the corresponding fields. You should also prove that you can successfully log in to **Boby** 's account using the new password.*

Activity 4: Countermeasures - Prepared Statement The root cause of the **SQL Injection** vulnerability is the application's inability to separate **code (commands)** from **data**. When constructing an **SQL statement**, the program (e.g. **PHP** code) knows what is data and what is command. However, when the SQL statement is sent to the database, the separation between data and code ceases to exist. This results in the commands that the **SQL interpreter** sees being different from the commands defined by the programmers. To solve this problem, the safest way is to use the **Prepared Statement** when developing programs.

To understand how a prepared statement prevents SQL Injection, we need to understand what happens when the database server receives a query (in the form of an SQL Statement). The **workflow** for executing a query is shown in *Figure 3*.

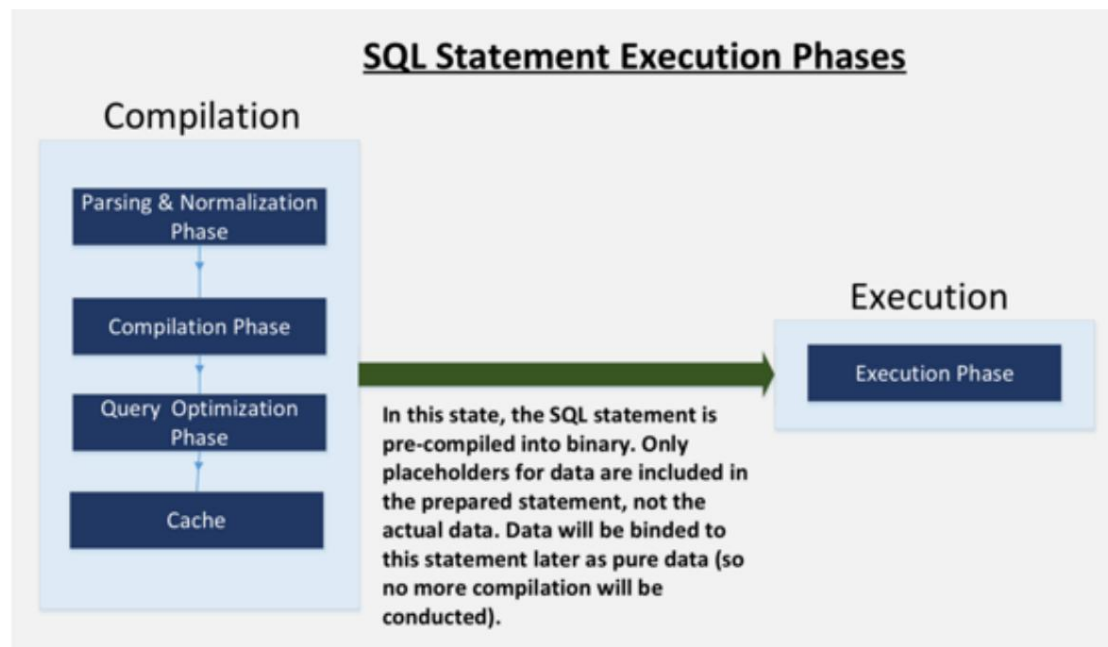


Figure 3. Workflow of an SQL Statement

In the broader stage of **translation**, queries first go through individual phases:

- *Analysis* and normalization phase , where a query is checked against the syntax and semantics.
- Compilation phase , where keywords (e.g. SELECT, FROM, UPDATE, etc.) are converted into machine language.
- Query *optimization* phase , where a number of different plans are calculated for executing the query, from which the optimal plan is finally selected.

- Temporary storage phase, where all designs that are ultimately selected are stored in the cache .

Whenever a new query arrives, the cache contents are first checked. If the query is already compiled in the cache, the *parsing*, *compilation* , and *optimization* phases will be skipped. Finally, the compiled query is passed to the **execution stage** where it is actually executed and returns the results.

The prepared statement **enters** this flow

actions at the compilation stage. A prepared statement will go through all the compilation phases and will be converted into a pre-built query **with blanks** for the data. In order for this pre-built query to be executed,

data must be added (**bind**) **at some point**, but this data **will not go through** the compilation step. Instead, it is **directly** bound to the pre-built query and sent to the execution engine. Therefore, even if there is SQL code in the data, since it will not go through the compilation step, the server will simply treat it **as part of the data**, without any special meaning. This is how the prepared statement prevents **SQL Injection attacks**.

Here is an example of how to use a prepared statement in

PHP. In its original form, the code that prepares the SELECT statement is vulnerable to SQL Injection attacks:

```
$sql = "SELECT name, local, gender
      FROM USER_TABLE
      WHERE id = $id AND password ='$pwd' ";
$result = $conn->query($sql)
```

Using the *prepared statement mechanism*, the same code can be rewritten as shown below

```
$stmt = $conn->prepare("SELECT name, local, gender
                      FROM USER_TABLE
                      WHERE id = ? and password = ? ");
// Bind parameters to the query
$stmt->bind_param("is", $id, $pwd);
$stmt->execute();
$stmt->bind_result($bind_name, $bind_local, $bind_gender);
$stmt->fetch();
```

Essentially, we divide the process of sending the SQL Statement to the database server **in two steps**:

- The first step is to send **only the code part**, i.e. a SQL Statement **without** the actual data. This is the preparation step. As we can see from the above code snippet, the actual data is replaced by question marks (?).

- In the next step, we send **the data** using the **bind_param() method**. The database server will treat all the elements sent in this step **only as data** and not as code and will match them to the question marks (?) of the prepared statement. In the **bind_param() method**, the first argument "i" indicates the types of the parameters: "i" means that the data in the variable \$id is of type *integer* while "s"

means that the data in the \$pwd variable is of type *string*.

Considering the above, you should complete the following activity:

The purpose of **Activity 4** is to use the prepared statement mechanism to **fix** the SQL Injection vulnerabilities in the application code that you have exploited in the previous activities. To do this, you will need to edit the **php code** in the files:

- **unsafe_home.php**, and
- **unsafe_edit_backend.php**

located in the **/var/www/SQLInjection** directory.

After making the fix, you should try some of the previous attacks to check whether you can still exploit the vulnerability or not.

In your report, you should explain how (the changes you made to the code) you fixed the specific files. You should also include screenshots of the attacks being executed after the fix to demonstrate the success of the fix.

Submission

You must submit a detailed **lab report** (in PDF) covering all the steps you took in each activity, your results and observations. Use screenshots and code snippets in your description, according to the instructions for each activity.